

EDBooks: AI-Enhanced Interactive Narratives for Programming Education

ANONYMOUS AUTHOR(S)

Large Language Models (LLMs) have shown the potential to be valuable teaching tools, with the potential of giving every student a personalized tutor. However, one challenge with using LLMs to learn new concepts is that when learning a topic in an unfamiliar domain, it can be difficult to know what questions to ask. Further, language models do not always encourage “active learning” where students can test and assess their understanding. In this paper, we propose ways to combine large language models with “traditional” learning materials (like e-books) to give readers the benefits of working with LLMs (the ability to ask personally interesting questions and receive personalized answers) with the benefits of a traditional e-book (having a structure and content that is pedagogically sound). This work shows one way that LLMs have the potential to improve learning materials and make personalized programming education more accessible to a broader audience.

ACM Reference Format:

Anonymous Author(s). 2024. EDBooks: AI-Enhanced Interactive Narratives for Programming Education. 1, 1 (May 2024), 23 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Dialogic learning emphasizes interactive, learner-centered approaches where students actively co-construct knowledge through dialogue [62]. Dialogic learning can be used for a wide variety of subjects, including computing education. Prior work has shown many benefits of dialogic learning [58], including increasing students’ engagement, encouraging active learning [63], critical thinking [15], and providing personalized learning experiences. However, providing dialogic learning at scale remains challenging [49]. Dialogic learning requires that instructors actively engage students, respond to their questions, and provide meaningful feedback. This is especially important in programming education, which requires active hands-on practice and feedback to build skills. Traditional classrooms can offer effective teaching and support but often lack enough instructors to for personalized feedback. Non-traditional environments like online courses and Self-Directed Learning (SDL) can provide scalable instruction but may lack personalized guidance and meaningful practice.

Although dialogic learning is primarily focused on synchronous, interactive educational experiences, the principles can also be applied to static educational content. For example, Daniel Friedman authored a series of books that teach computing concepts through a dialog between a learner and an instructor [13, 14]. However, prior approaches [13, 59] using the question-and-answer format to write programming learning materials have limitations in providing truly adaptive and interactive content tailored to diverse learners’ needs. Thus, the question that motivates this research is: **how can we enable high-quality, personalized, and interactive dialogic learning experiences at scale?** We are particularly focused on supporting dialogic learning for *asynchronous* programming education, which can include SDL or assignments that are part of a traditional course.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 Association for Computing Machinery.

Manuscript submitted to ACM

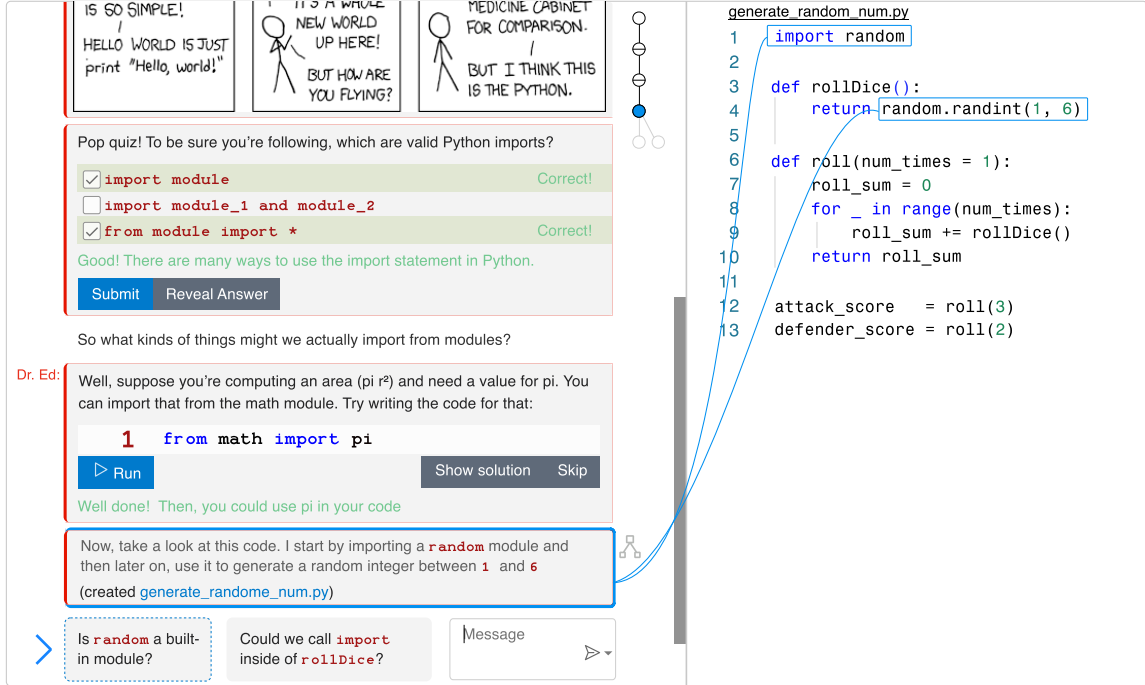


Fig. 1. An illustration of EDBook content that describes the basics of Python modules. Readers learn through an interactive dialog (left column) that includes a natural language didactic dialog, multiple choice questions, and code writing questions. With EDBook, readers work through this narrative over time. At any point in the narrative, they can also interact with a context-aware LLM for additional questions. These dialogs can also include larger code samples that are incrementally built and explained over the course of the narrative (right column). Deictic pointers (represented as blue lines) can link the narratives with specific parts of code. At the bottom of the leftmost column, the reader is given three options for how to proceed with the narrative: two pre-built questions (“is random a built-in module?” and “Could we call import inside of rollDice?”) and the option to query an LLM.

Recently, the advent of LLMs such as ChatGPT has opened new possibilities for developing engaging and personalized educational content that can provide responsive guidance at scale [21, 39]. LLMs can accurately respond to a wide variety of natural language questions and requests. However, one challenge with using open-ended LLMs to learn new concepts is that it can be difficult for learners to know what questions to ask to effectively build understanding in an unfamiliar domain. Although many *task-oriented* LLM extensions (such as chatbots for booking flights) can naturally elicit information from users (e.g., departure airport, destination, etc.), they do not have a way to ensure readers understand educational content. Ideally, learners should have the flexibility to explore and ask questions on their own while achieving larger learning goals, which could be specified by instructors and field experts. This highlights a key challenge: **how can we enable programming learners to benefit from interactive LLMs while aligning interactions with pedagogical goals?**

In this paper, we propose a platform that addresses these challenges by enabling a limited form of dialogic learning. We designed EDBook¹ a new platform for shareable, interactive educational content². With EDBook, content authors can write dialog trees—dialogs where messages can have multiple responses—that are aimed towards a learning goal.

¹“EDBook” is short for **E**lectronic **D**ialogic **B**ook

²Readers are invited to try an anonymized deployment of EDBook at <https://edbook.net/>.

These dialogs can involve any number of ‘participants’ but typically contain at least an instructor persona who guides readers through didactic dialog and a persona representing the reader. They can contain interactive code-writing questions to give readers hands-on practice, multiple-choice questions to help readers assess their understanding, larger code samples that are incrementally built and explained over the course of the dialog, and multimedia, as Figure 1 shows. Readers can interact with a pre-made dialog tree or can self-explore through open-ended interactions with a context-aware LLM. Readers are guided towards pre-determined learning goals, which are represented by a ‘target’ point in the dialog tree—typically a leaf node that represents the end of the dialog.

Contributions: The key innovation of this work is in enabling dialogic learning experiences that are open-ended yet goal-aligned. By doing this, we can guide students towards learning goals while providing responsive guidance and reinforcing concepts through interactive practice. To our knowledge, this represents the first system to combine these techniques for programming instruction.

The key contributions of this work are:

- The design of EDBook, the first platform to integrate pre-written dialogic narratives with open-ended context-informed LLMs. EDBook also contains additional features (like code writing questions and incremental code building) specifically designed for programming education.
- Techniques to align open-ended LLM interactions with larger learning goals, enabling learner agency while meeting pedagogical objectives.
- A representation for branched dialogs, code examples, and learner state that maintains readability and understandability for learners.
- A study evaluating learning outcomes and qualitative experiences using EDBook for programming education.

The EDBook platform also includes several additional features that are not entirely novel but that are complimentary to its design, including (1) interactive quiz questions (including code-writing questions and multiple-choice questions) that help students assess their understanding of the material, (2) the ability to incrementally build up and explain large code samples over the course of a narrative, and (3) deictic code pointers that tie narratives with the larger code samples they discuss. Although these features are not novel independently, we show how they can be effectively integrated into the larger EDBook platform and dialogic learning more generally. Overall, our results highlight the promise of dialogic learning with LLMs for increased engagement, agency, and flexible learning tools that scale.

2 RELATED WORK

Steve:TO ADD: Scripted Vicarious Dialogues: Educational Video Augmentation Method for Increasing Isolated Students’ Engagement [56] Steve:TO ADD: CHATA: Towards an Intelligent Question-Answer Teaching Assistant using Open-Source LLMs [22] Our work on the EDBook platform builds on a rich body of prior work on LLMs in education, textbook platforms, techniques for connecting code with dialog, and educational narratives.

2.1 Using Artificial Intelligence (AI) and Large Language Models (LLMs) as Learning Tools

Researchers and practitioners have recognized the value of Artificial Intelligence (AI) as a learning tool [21, 23], particularly for Self-Directed Learning (SDL) [39]. Prior work has integrated AI and dialog systems into education in a variety of ways. Conversational agents and chatbots have been used to support personalized learning experiences through

natural dialog [19, 36, 42]. Intelligent tutoring systems can adaptively provide hints and feedback to learners, similar to human tutors [25]. They can also be particularly useful for learning new natural languages [3]. These systems demonstrate the potential for AI to enable responsive and adaptive education.

More recent work on AI in education has also examined Large Language Models (LLMs) in particular. LLMs can answer student questions [23, 28, 38], generate explanations [33], and even solve many coding problems [29, 48]. However, if not constrained properly, LLMs may produce misleading, incorrect, or educationally unproductive responses [18]. Effectively aligning LLMs interactions with learning goals remains an open challenge.

Within this context, platforms like Graphologue [26] and Sensecape [54] both explore ways to leverage LLMs to arrange learning materials. However, Graphologue’s design is oriented towards building a deep understanding of one particular concept by exploring multiple levels of abstraction for LLM responses and Sensecape’s design is oriented towards sensemaking. Neither is focused on longer educational narratives, as the EDBook platform is. The features of Graphologue could be combined with the EDBook platform to produce narratives that have more ‘local’ detail throughout.

One feature of the EDBook platform is that it can represent and help authors automatically generate dialogic content between any number of agents, based on descriptions of those agents. Commercial tools like character.ai [57] also allow users to create conversations with simulated agents. Similarly, Markel *et al.* explored how LLMs can simulate students to help train instructors [43]. Unlike these platforms, EDBook dialogs are designed to guide readers towards targeted learning goals.

2.2 The Impact of LLMs on Programming Education

Since modern LLMs like GPT-3 have proven to be capable of solving a variety of problems [5]—particularly for programming problems—researchers have recognized that LLMs are likely to have a profound effect on almost every aspect of programming education [2]. This includes how we assess understanding, how we generate practice exercises [41], how we create programming tutorials [18], and what is pedagogically important to teach (e.g., perhaps de-emphasizing low-level implementation work that can be automated). Prior work has studied many aspects of the impact of LLMs on programming education. Kazemitabaar *et al.* found that code generation tools can be a valuable learning tool in introductory programming education and that using code generation tools in the context of learning could reduce frustration without negatively impacting students’ understanding [29]. Sarsa *et al.* found that LLMs can also generate high-quality coding exercises that can be directly used by instructors in their courses [51]. Additionally, the code explanations produced by LLMs also demonstrate a high level of correctness, making it a promising tool with the potential to expedite tutoring processes for teaching assistants [33, 51].

For novice programming students, the incorporation of LLMs could potentially reframe their educational objectives. There arises a heightened emphasis on acquiring the proficiency to explain and communicate algorithms, as it directly influences the accuracy of code produced by LLMs [37]. In addition, the significance of code debugging and extending should also be emphasized, given that LLMs is capable of generating basic code for learners [2]. By leveraging the capabilities of LLMs, students can more easily understand error messages, identify mistakes, and accelerate their overall learning progress [34].

The incorporation of LLMs into programming education also raises certain concerns. Code generated by LLMs can exhibit biased structures and comments [47] and may also contain insecure vulnerabilities [6]. Furthermore, Becker *et al.* suggest that the coding style and approaches adopted by LLMs might prove too advanced for novice programmers to readily adopt [2]. In this case, EDBook aims to create a semi-structured learning environment, enabling learners

to engage with LLMs under appropriate, dialogue-style guidance. This approach is designed to mitigate the potential negative impacts that could be transferred to learners.

2.3 Interactive Textbook Platforms

There are several interactive textbook platforms, particularly for teaching programming. Runestone [11], a platform on which several widely-used programming textbooks are built, enables interactive content, including code-writing questions, multiple choice, and Parsons problems [10] (where students drag and drop code blocks to arrange them in a correct order rather than writing code manually). Jupyter Book [7] is another computational platform that empowers the creation of interactive textbooks. It integrates narrative text, code execution, visualizations, and interactive widgets within the environment. Al-Gahmi *et al.* emphasize the effectiveness of using Jupyter as a valuable tool in computing education within the classroom, leading to significant improvements in students' performances on assignments [1]. OpenDSA [52] is an open-source platform designed to streamline the creation of interactive textbooks for computer science-related subjects. It enables instructors to create visualizations, quizzes, and animations within the web-based book, enhancing the learning experience for students. Prior studies indicate that many students prefer interactive textbooks over conventional text materials [12, 53]. However, existing interactive textbook solutions have certain limitations. Computational notebooks such as Jupyter Notebook might not be intuitive for novice learners [27]. On the other hand, other web-based textbooks can lack support for complex code execution and note-taking [44]. Consequently, students need to use multiple platforms simultaneously. The EDBook platform is designed as an extension of the Visual Studio Code platform. By integrating learning and coding within a single interface, EDBook aims to help students engage with content and practice coding cohesively.

2.4 Goal-Oriented Dialog Systems and State-Based Representations of Dialogs

Goal-oriented dialog systems are dialog systems that give users agency and choice while still directing them towards a set goal. Goal-oriented dialogue systems are commonly applied to chatbots [17, 64], which are developed to provide real-time assistance to users in accomplishing specific tasks. Prior studies indicate that numerous goal-oriented dialogue chatbots struggle to meet users' expectations [24, 65]. This challenge arises due to the intricacies of automating certain tasks [8], the complexities of dialogues [17], and the inherent difficulty in accurately identifying natural language content [35]. Most systems are targeted towards goals that are specific and concrete, such as booking a flight or scheduling a meeting [45, 50]. By contrast, notebooks in EDBook have the goal of building conceptual understanding for readers. To mitigate the challenges presented by more open-ended goal-oriented dialog systems, EDBook adopts a tree-based scripted dialogue style for its notebooks to minimize dialogue variability. Additionally, the preset message options can also reduce potential interruptions during dialogues [35]. In EDBook, the learning objectives can be considered "abstract tasks". To aid readers in identifying their "task progress", EDBook incorporates quizzes within its notebooks. Many tools, including Dialogflow [40] and Tae *et al.*'s work [30], use state-based representations of dialogs. This feature supports users to go back to the previous chat boxes and modify any cell to their favor. Extensive research has also been done in predicting future utterances from textual data, such as example-based dialog systems [32]. Example-based dialog systems can generate system responses for user input based on stored data examples.

After we have considered possible visualized dialog representations from traditional spoken dialogs [9], we finally decide to use representations of dialogs that are both state-based and tree-based. Notably, our work is designed and implemented as a tool to help users in navigating programming instructional content. It serves the purpose of guiding users through the proposed learning path while also alerting them if their progression diverges from the initially

intended learning trajectory. This is achieved by a seamless amalgamation of the state-based and tree-based representations. Moreover, the system enables users to embark on exploratory journeys, empowering them to attempt different paths within the instructional dialogue. If users acquire new insights during their exploration and decide to revise their approach, the system facilitates a fluid return to previous conversations.

Topic shifting [55]—shifting open-ended conversations towards a target topic—is also a promising approach. However, existing topic-shifting approaches are designed for conceptually simple topics (single entities, like ‘football’) [55] rather than complex learning goals. Further, topic shifting relies on LLM-generated content, which can be inaccurate or misaligned with learning goals.

2.5 Connecting Code and Dialog

chat.codes [46] is a tool for discussing code. Although the use case of EDBook is different, there is overlap in the features of EDBook and chat.codes. Like chat.codes [46], EDBook enables programmers to create deictic references between messages and specific regions of code. However, EDBook’s design of these deictic references differs from that of chat.codes in several important ways. First, deictic references in chat.codes needed to be specified on the *character* level through markdown annotations. For example: “[these lines](code.js:L23-L42)” creates a deictic reference in chat.codes. If the user hovered over “these lines”, lines 23–42 in code.js would be highlighted. In our experimentation with writing content, we found that we rarely needed to write fine-grained references. Thus, deictic references in EDBook are made at a *cell* level—cells contain a list of references, rather than specific parts of text. We found that this change simplified deictic pointers for both authors and readers. For authors, EDBook reduces the amount of syntax they need to learn and allows them to add deictic references through simpler point-and-click interactions³. For readers, EDBook’s representation of deictic pointers allows them to see them automatically. Unlike in chat.codes, where readers needed to actively hover over the relevant text to see pointers, EDBook can always display deictic references for the selected cell(s). Further, EDBook visualizes a curved line between the selected cell(s) and the code they discuss, which helps form a clearer connection between the narrative and code.

Callisto [61] also contains deictic references, in the context of the Jupyter Notebook platform. Callisto’s deictic references are also coarser than those of chat.codes, with Like EDBook, Callisto’s references are made on a cell-level, where relevant code will be highlighted when one message is selected. The primary distinction between Callisto and EDBook lies in their approach to highlighting code for deictic references. In Callisto [61], a highlight color is used to emphasize relevant content, including not only code but also notebook cells, markers, snapshots, and different versions. This highlight color strategy maintains a uniform experience across various content types, effectively highlighting the contextual discussion of selected messages. In contrast, the EDBook utilizes a curved line to connect relevant code and cells. This method is designed primarily for code referencing, which creates a more intuitive connection between code and narrative cells.

Colaroid [60], another extension for Visual Studio Code, also combines narrative text with iterative code changes for authoring coding tutorials. Like EDBook, Colaroid enables authors to explain larger pieces of code through a series of steps. However, there are several key differences between Colaroid and EDBook. First, Colaroid does not contain any *dialogic* features—readers cannot interrupt the narrative to query an LLM and authors cannot build assessments (e.g., multiple-choice or coding questions) into their narratives. Second, Colaroid does not include deictic references to

³chat.codes includes User Interface (UI) functionality for adding deictic references without typing out the syntax manually. Users can highlight a section of an un-sent message and then highlight a region of code to add a reference to the highlighted part of the message. However, we found that this interaction was not discoverable and could be difficult for authors, particularly when they made subsequent modifications.

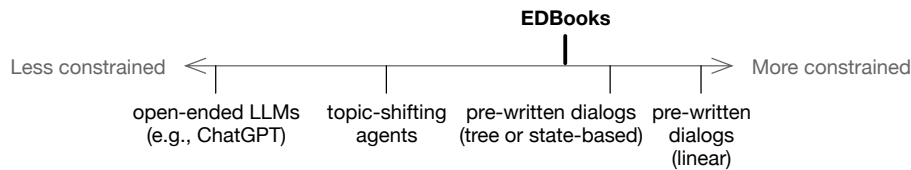


Fig. 2. An illustration of different dialog system types along a continuum from less constrained input (left) to more (right).

better tie the narrative text with content. Finally, EDBook gives users more agency in choosing how to follow a given tutorial. Readers can select which response they want to give and can write their own parts of narratives. However, Colaroid also has several features that EDBook would benefit from—most notably, the ability to integrate with git and version control tools and the ability to re-play user interactions on a UI.

Like the EDBook platform, Torii [20] also allows code examples to be built up incrementally. However, beyond this similarity, there are significant differences between the design and use cases for these two tools. EDBooks focus more on explanation, exploration, and learning through interactive dialog whereas Torii is designed to give readers clear walkthroughs of a single codebase.

3 EDBOOK DESIGN

We divide our discussion of the EDBook design into two. First, we describe the core mechanics of EDBook. Then, we describe how complementary features, like interactive questions integrate with the EDBook platform. Finally, we conclude by describing the aspects of EDBook that are novel and unique relative to prior work.

3.1 Core Design and Novelty

The core design of the EDBook platform is in how it combines dialog trees with open-ended LLM interactions. The fundamental design questions are about balancing the ability for learners to explore while meeting concrete learning goals and how to represent dialogs in a way that is coherent and understandable.

3.1.1 Balancing Free Exploration with Concrete Learning Goals. One of the core design questions is how to balance concrete learning goals with open-ended exploration. Figure 2 illustrates where various dialog system techniques stand on a continuum of degree of constraint. Less constrained systems (such as ChatGPT and other open-ended LLMs) can give learners more flexibility and engage them to direct their own learning. However, they cannot guide learners towards larger learning goals and might provide information that is inaccurate. On the other end of the spectrum, completely scripted dialog systems (such as pre-written linear dialogs [13, 14]) can ensure that learners meet pre-determined learning goals and give accurate information. However, they can also limit learner agency and creativity.

EDBook aims to strike a balance. It uses pre-written dialog trees (which can be guaranteed to meet pre-determined learning goals with accurate information) while allowing open-ended LLM queries. In essence, the dialog trees act as “rails” to keep learners on track, but students can go “off-road” by asking their own questions. These “rails” can also help readers ensure that they are on the right path to achieve the target learning goals. Providing both dialogic structure and agency represents a novel approach to aligning open-ended LLM interactions with pedagogical goals. Future advances in LLM accuracy, alignment, and goal planning might enable less constrained systems in the future.

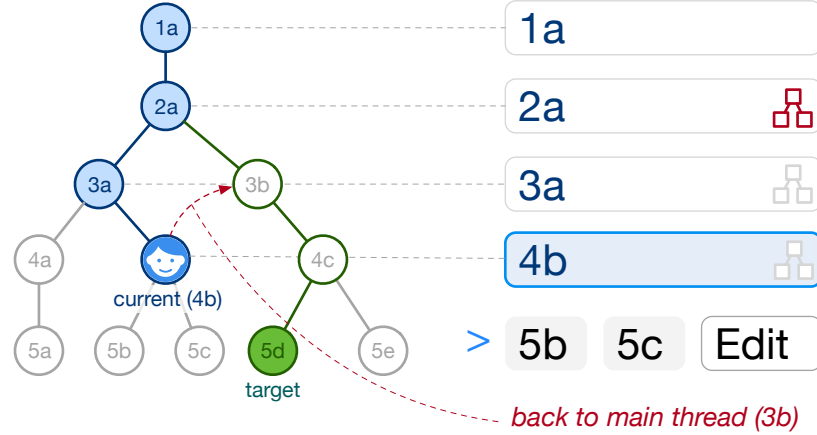


Fig. 3. Conceptually, EDBook dialogs are trees (left) but these trees are visually simplified and flattened in the reader UI (right) for readability. On the left is an illustration of an example dialog tree. Each node in the tree represents a message and is labeled according to its depth (1–5) and breadth-first search order (a–z). For example, 3a represents the leftmost node in the third layer. EDBook allows instructors to define a ‘target’ node (message 5d in this example) that represents completion of the learning goals for the page. In this example, the user is currently focused on message 4b, so ‘ancestor’ cells 1a, 2a, 3a, and 4b are visible in the UI (right). In the UI, users can also select pre-existing subsequent messages (5b and 5c) or write a custom message (‘Edit’) that will query an LLM to add a response message. The user’s current node is part of the dialog tree but not on the path of the target, 5d. Thus, the user is also nudged to go to a node that would move them back towards target node 5d by back-tracking to node 3b (with a “back to main thread” button). The UI also shows icons (db) to indicate messages with multiple possible responses (right) and calls attention to the first message where the user’s path diverges away from the target message (in red db).

3.1.2 Dialog Representation. The next fundamental question is how to represent dialogs while ensuring learners meet pre-determined learning goals. An essential criterion is that *any information related to the learning goals must be accurate*. Despite their capabilities, LLMs can still generate inaccurate information. Thus, any content related to the core learning goals should be either written or verified by EDBook authors. This is especially important for assessments (e.g., coding and multiple-choice questions), where we anecdotally found that LLMs often wrote assessments that were *accurate* but missed the primary learning goals.

Because EDBook authors are responsible for verifying or writing learning material, dialog representations with large state spaces (e.g., state-based representations) or that are not predictable (e.g., using topic-shifting or open-ended dialogs) would not be appropriate. We instead designed EDBooks to represent dialogs as trees, with a single ‘target’ leaf node that represents completion of the learning goals. Each node in the tree represents a dialog message, where the root node is the starting prompt, and child nodes are possible responses. This representation of dialogs reduces the amount of information that authors need to verify (only the nodes from the ‘root’ to the ‘target’). Using dialog as trees rather than linear scripts also supports dynamic narratives with user agency.

For readers, these dialog trees are simplified and flattened to ensure the dialog is coherent and readable. As Figure 3 illustrates, readers can see their current node and all its ancestors, along with icons to indicate where there are multiple options to explore. Readers can explore tangents before being guided back to the target learning path. As readers progress, the UI highlights divergence from this target path, with a link to go “back to the main thread”—to the first node that branched away from the target node.

3.1.3 Using and Managing LLM-Generated Content. A key design feature of EDBook is in the way that it integrates LLMs with pre-written narratives. Both authors and readers can use LLMs in EDBook. Authors can use LLMs to generate an initial dialog tree. They first specify a list of participants (by default, one ‘instructor’ persona, named *Dr. Ed* and a persona representing the reader). Each of these personas includes a description, which can be supplied to the LLM to generate appropriate dialog. Most importantly, the description of the instructor persona should be aligned with the desired learning goals—for example, “A knowledgeable instructor teaching the basics of for loops in Python in a way that is suitable for novices.”. Authors can then either generate dialogic content turn-by-turn—for example, by adding messages on behalf of the ‘student’ persona and letting ChatGPT respond as the ‘instructor’ persona. Authors can also leverage ChatGPT to generate multi-turn dialogs, by describing the dialog topic and the participants’ personas. In both cases, authors can manually edit the LLM-generated content as necessary. In our experimentation, we found mixed results with fully LLM-generated content. When asked about popular topics, it generated content that was largely accurate but the generated messages were not always engaging for readers (for example, without further guidance, messages would often be longer than what we found most readers would prefer) and the generated assessments were not always aligned with the learning goals (for example, there might be multiple choice questions focused on terminology rather than core conceptual ideas).

EDBook readers can engage with LLMs at any point in the dialog, as necessary. Whenever they query an LLM, the current node and all ancestor nodes are passed in as context. The LLM’s response then creates a new branch in the dialog tree. By default, the LLM’s response is associated with the ‘instructor’ persona (Dr. Ed) but any LLM-generated content is also accompanied by a warning that the content is AI-generated and has not been verified, as Figure 4 illustrates.

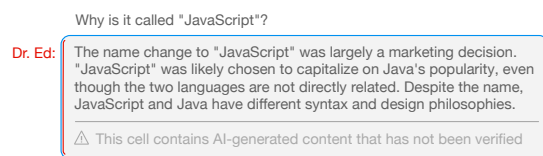


Fig. 4. Warning labels are added to any cells that contain unverified LLM-generated content.

3.2 Complementary Features

EDBook also contains many features that complement its core interactions. These features are not novel individually but are crucial for the effectiveness of EDBook as a learning tool.

3.2.1 Incremental Code Changes. Many instruction materials contain larger code samples as part of the learning material. Because these code samples are meant to be instructive, they should be constructed in a way that makes them as understandable as possible. As prior work has found, tutorial authors often want to incrementally build didactic code examples and explain these code samples in an order that is different from the “code order” [20].

To support this, EDBook allows authors to build up larger code examples incrementally over the course of a narrative. Each message can be associated with a textual code diff representing additions, removals, or unmodified lines relative to prior code versions. These diffs are rendered to visually show the code evolving across messages. This allows authors to introduce concepts through

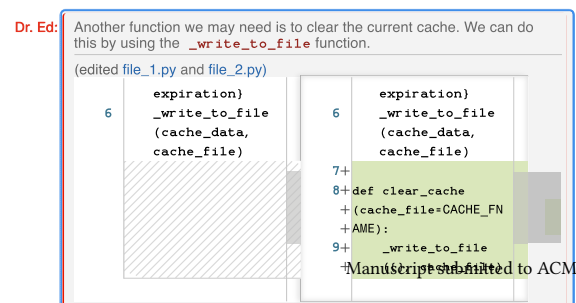


Fig. 5. Instructors can work with larger codebases that change over time. Readers can see these incremental code edits, as diffs.

iterative code refinements. EDBook also displays code changes inline with messages, as Figure 5 illustrates.

3.2.2 Deictic References. It is important to be able to orient readers to understand code samples as they are being incrementally built. But it can be difficult for readers to understand which message refers to which part of code, particularly for larger code samples. Prior work [46, 61] addresses this through ‘deictic’ references—visual indicators that associate messages with the part of code it is discussing. Similarly, EDBook allows authors to create deictic pointers, which are displayed as curved lines between messages and regions of code, as Figure 1 illustrates.

3.2.3 Interactive Assessments. To check and reinforce reader comprehension, authors can incorporate interactive assessments into EDBook narratives. Currently, EDBook supports multiple-choice questions and code writing challenges. These assessments provide opportunities for active learning and feedback. Learners can validate their knowledge through practice. Incorrect answers reveal areas needing more focus.

3.3 Novelty and Uniqueness

As far as we are aware, EDBook is the first platform to combine pre-written dialog trees with context-informed LLMs. This design mitigates unhelpful LLM responses and keeps learners focused while allowing flexibility. The dialog trees provide goal-oriented structure that we can guarantee to be accurate while the LLM queries enable personalized explorations. The technique is novel in using dialog state for education-oriented LLM guidance. Further, the design of EDBook’s deictic references (visually tying messages with the code being discussed) is unique relative to other tools that contain deictic code references [46, 61].

Finally, complementary capabilities (incremental code edits, deictic references, and interactive assessments) enrich the core dialogic learning in EDBook. They demonstrate how interactive features can be effectively integrated to engage learners. The combination enables more pedagogically productive programming narratives.

4 USER STUDY

To evaluate the effectiveness of EDBook and identify usage patterns, we conducted a within-subjects user study with 20 programming learners. Our study focused on the *reading* experiences of learners, as opposed to the authoring experience. We view the reading experience as disproportionately important because we expect many more users to be readers than authors.

4.1 Recruitment

We reached out to students enrolled at (*INSTITUTIONS REMOVED FOR ANONYMITY*) through email. Participants were required to have taken at least one Python programming course but no prior experience with the specific content covered in the tutorials—no experience with Scheme or LISP (one task in our study) and no experience with writing code for caching web requests in Python (another task in our study). We determined eligibility by participants’ responses on a screening form.

4.2 Participants

Of the eligible participants, 20 completed the study. We selected participants based on their eligibility and aimed for a diverse set of backgrounds. One additional participant began the study but stopped the study after performing one task due to internet connectivity problems. All participants were students, as learners are the primary audience for EDBooks. Most participants (16 of 20) were in degree programs that involved programming (either Computer Science or Information), as we required some prior technical knowledge. Relevant demographics of the 20 remaining participants are itemized below:

Participant #	Age	Gender	Education Program	Prog Experience	LLM Usage
1	18–24	Woman	Undergraduate	Less than one year	Rarely
2	18–24	Woman	Master's	1–2 years	Occasionally
3	18–24	Man	Undergraduate	1–2 years	Occasionally
4	18–24	Woman	Undergraduate	1–2 years	Occasionally
5	25–34	Woman	Ph.D.	3–5 years	Frequently
6	18–24	Man	Undergraduate	Less than one year	Rarely
7	18–24	Woman	Master's	Less than one year	Occasionally
8	18–24	Woman	Undergraduate	1–2 years	Frequently
9	25–34	Woman	Master's	Less than one year	Occasionally
10	18–24	Woman	Undergraduate	6–10 years	Rarely
12	25–34	Woman	Master's	3–5 years	Frequently
13	18–24	Woman	Master's	3–5 years	Rarely
14	25–34	Man	Master's	6–10 years	Frequently
15	18–24	Man	Master's	1–2 years	Frequently
16	18–24	Man	Undergraduate	1–2 years	Rarely
17	25–34	Man	Ph.D.	3–5 years	Occasionally
18	25–34	Man	Ph.D.	1–2 years	Occasionally
19	18–24	Woman	Master's	Less than one year	Frequently
20	18–24	Woman	Undergraduate	1–2 years	Occasionally
21	18–24	Woman	Ph.D.	1–2 years	Rarely

Table 1. Participants' Data. Participant 11 did not complete the study and is thus not included in this table or our analysis.

All participants had prior experience using ChatGPT and 15 of 20 had used ChatGPT in their own learning, primarily for programming support. All but one participant had used Visual Studio Code.

4.3 Setup and Procedure

Our study had two conditions, illustrated in Figure 6:

- Condition 1 (C_{EDBook}): Participants read educational dialogic EDBook content.
- Condition 2 ($C_{\text{Web+GPT}}$): Participants read the same educational content but in non-dialogic format and translated into a standard webpage format. They also had access to ChatGPT, with the content provided as background context to ensure it produced responses that were consistent with the content.

And two tasks:

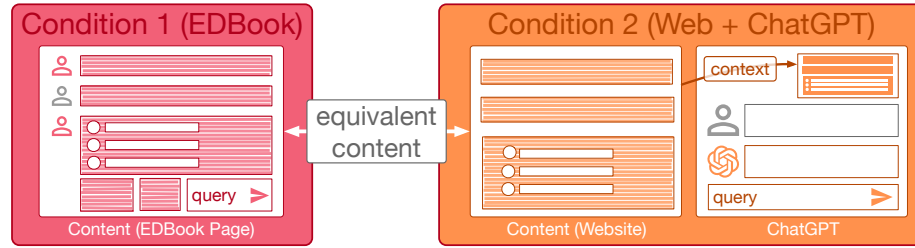


Fig. 6. An Illustration of the two conditions in our evaluation of EDBook. In C_{EDBook} (left), participants performed a task with the EDBook platform. In $C_{\text{Web+GPT}}$ (right), participants performed a task with equivalent content, converted to a webpage. $C_{\text{Web+GPT}}$ participants can also interact with a “context-informed” instance of ChatGPT that has the context of the relevant webpage content to provide contextually-appropriate responses.

- Task 1 ($\mathcal{T}_{\text{scheme}}$): In this task, participants learned the basics of the Scheme programming language (a functional language that is a dialect of Lisp). This task was designed to represent the challenges of *learning content that is conceptually new*, as most participants have no prior familiarity with any form of declarative programming.
- Task 2 ($\mathcal{T}_{\text{cache}}$): In this task, participants walked through how to write Python code that makes web requests (using the requests module) and caches the result for a pre-specified period of time. Participants had familiarity with the underlying programming language (Python) but not with this specific application or algorithm. This task was designed to represent the challenges of *learning through building*. Although none of the underlying concepts were new to participants, the specific way they were put together in a larger (approximately 70-line) code sample could be difficult to follow.

In both conditions, tasks were split into smaller topic-focused chapters. These chapters included quiz questions and other methods to allow readers to check their understanding. These quiz questions were different from the assessment questions we used after each task to assess their understanding. In both conditions, participants interacted with OpenAI’s ChatGPT-3.5.

Figure 7 illustrates our study design. The study coordinator began each study by asking participants to complete a consent form and informing them of the study procedure. They then completed two learning tasks ($\mathcal{T}_{\text{scheme}}$ and $\mathcal{T}_{\text{cache}}$) in both conditions (C_{EDBook} and $C_{\text{Web+GPT}}$), in a latin square design to counter order effects. Before each task, participants spent approximately 10 minutes walking through the interface(s) they would interact with for that condition (instructions on how to use EDBooks or ChatGPT). After each task, participants completed assessments to gauge their understanding of the materials. They were given 10 minutes for each assessment and any incomplete answers were marked as “incorrect”. During these assessments, they were allowed to go back and check the learning materials but they were not allowed to send further queries to the LLM (to prevent participants from feeding assessment questions back to the LLM without demonstrating an understanding of the material). We also asked participants to fill out a short survey after each task. After participants completed both tasks, the study coordinator conducted a post-task interview.

All study sessions were conducted virtually. With participants’ approval, the study coordinators recorded their screens and collected other usage telemetry information for subsequent analyses. Each session took approximately two hours and participants received \$50 (USD) as compensation. Our institution’s Institutional Review Board (IRB) approved our study setup.

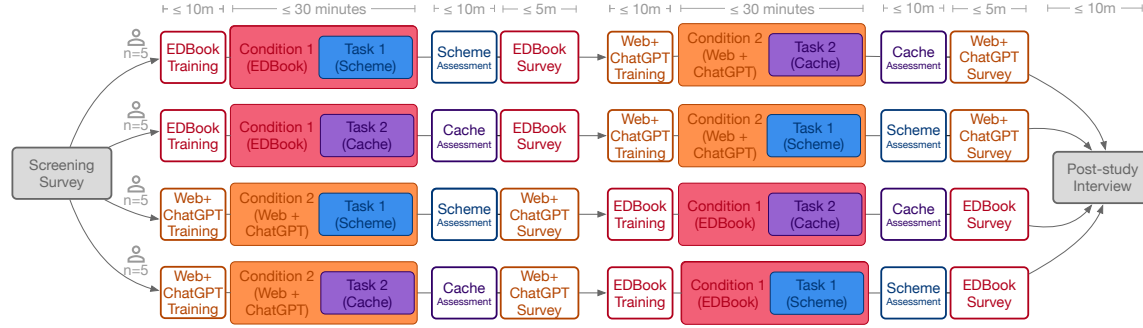


Fig. 7. An Illustration of the steps that participants take in our study. Each participant performs two tasks using two different conditions. They complete post-task surveys and interviews.

4.4 Results

We describe our quantitative results below and summarize our interview responses and findings in section 4.5.

4.4.1 Measures of Learning Activities. As Table 2 shows, we collected several measures of the learning activities, including the number of questions participants asked with LLMs, the time they spent on learning, the time they spent on assessments, assessment scores, and the number of attempts to try example code. We found that participants in the C_{EDBook} spent significantly longer time with the learning materials ($\bar{x} = 1817.85, \sigma = 265.22$) than in the $C_{\text{Web+GPT}}$ ($\bar{x} = 1389.90, \sigma = 461.18$). In addition, participants in the C_{EDBook} ($\bar{x} = 0.95, \sigma = 0.13$) made more attempts to answer the quiz questions and experiment with the example code than those in the $C_{\text{Web+GPT}}$ ($\bar{x} = 0.40, \sigma = 0.36$).

4.4.2 Types of Questions Asked with LLMs. As Table 3 shows, we manually coded the questions that participants asked with LLMs. We used Bloom’s Taxonomy [4] for closed coding, including six levels—remember, understand, apply, analyze, evaluate, and create. In both conditions, we did not observe questions related to remembering (recalling facts and basic concepts). In C_{EDBook} , participants asked more questions for seeking usage of a concept in new situations (applying) and differentiating between two concepts (analyzing), whereas in $C_{\text{Web+GPT}}$, participants asked more questions for seeking explanations of a code solution (evaluating) or directly requesting a solution for a coding problem (creating). This could indicate that EDBooks led participants to actively reflect on the concepts they learned. This reflection includes considering edge cases, evaluating generalizability, and connecting with previously learned concepts. In contrast, in $C_{\text{Web+GPT}}$, participants adopted a more passive learning approach, primarily seeking solutions to quiz questions from LLMs and seldom experimenting with these solutions within the provided Integrated Development Environment (IDE).

4.4.3 Post-task Questionnaire Results. As Table 4 shows, we examined the post-task questionnaire results. Among all the measures, we found that in C_{EDBook} , participants gave a significantly higher rating to the question “I was highly engaged in the tutorial when learning” ($p < 0.05$, ANOVA). Participants also reported that the code-writing questions are more helpful in C_{EDBook} than $C_{\text{Web+GPT}}$ ($p < 0.05$, ANOVA).

4.5 Interview Responses and Findings

Based on the results described above and interviews with participants, we identified several key findings.

Measures	Condition	$\bar{x}$	σ	
Number of Questions Asked with LLMs	$C_{\text{Web+GPT}}$	3.45	3.89	
	C_{EDBook}	3.30	3.45	
Time Spent on Learning* ($p < 0.01$)	$C_{\text{Web+GPT}}$	1389.90	461.18	
	C_{EDBook}	1817.85	265.22	
Time Spent on Assessment	$C_{\text{Web+GPT}}$	575.0	39.38	
	C_{EDBook}	543.6	100.91	
Assessment Score	$C_{\text{Web+GPT}}$	1.35	0.74	
	C_{EDBook}	1.50	0.76	
Attempts to Try Example Code* ($p < 0.001$)	$C_{\text{Web+GPT}}$	0.40	0.36	
	C_{EDBook}	0.95	0.13	

Table 2. We collected the number of questions participants asked with LLMs, the time they have spent on learning and the exit test, the test score, and the number of attempts to try quiz questions. (mean: $\bar{x}$, standard deviation: σ). Our comparison suggests that the time spent on learning is significantly different in the two conditions ($p < 0.01$, ANOVA); the number of attempts to try quiz questions is also significantly different in the two conditions ($p < 0.001$, ANOVA).

Code	Definition	Example	$C_{\text{Web+GPT}}$ vs. C_{EDBook}
Understand	Seeking explanation of a concept instead of a real coding problem	“What is private function in Python?”	
Apply	Seeking usage of a concept in new situation	“Can you perform multiple operations in one s-expression?”	
Analyze	Differentiating between two concepts	“Is write or display more similar to a print function in Python?”	
Evaluate	Explaining a coding solution (e.g., asking explanations of the answers)	“Why are there a lot of if statements in the code?”	
Create	Directly requesting a solution for a coding problem	(copy and pasted code writing question to get an answer)	
Others	Conversations instead of questions	“I’ve learned the concept of caching before, but not familiar with detailed info”	

Table 3. Types of Questions Participants Asked to LLMs

4.5.1 Participants used more time for learning the materials in C_{EDBook} than $C_{\text{Web+GPT}}$. As our results show, participants spent more time in C_{EDBook} than $C_{\text{Web+GPT}}$ on average ($p < 0.01$). Based on our observations and intuition, we believe there are several reasons for this. First, the EDBook platform requires at least some user interaction to reveal content, whereas $C_{\text{Web+GPT}}$ requires no interaction beyond scrolling. Second (and possibly consequentially), participants attempted more coding and multiple-choice exercises in C_{EDBook} than in $C_{\text{Web+GPT}}$, as we describe in 4.5.2. As we found in interviews, participants liked the ability to test out code immediately in C_{EDBook} , which might have led to increased engagement and more time spent on coding quizzes. Third, as section 4.5.6 describes, participants expressed that they felt more engaged in C_{EDBook} than in $C_{\text{Web+GPT}}$. This increased engagement might have led to more time spent in C_{EDBook} . Finally,

EDBooks were new to all participants and it takes some time to get acquainted with any unfamiliar tool, despite having a tutorial.

4.5.2 Participants attempted more coding quizzes in C_{EDBook} than $C_{Web+GPT}$. Our results also show that participants attempted more coding quizzes in C_{EDBook} than $C_{Web+GPT}$ ($p < 0.001$). Again, this might be because EDBooks require user interaction to advance the narrative. This means that participants needed to engage with coding question at some level (the need to explicitly decide to “skip” the quiz or reveal the answer). It might also be because the EDBook platform integrates tutorial content with a code authoring and testing environment:

“[With EDBooks,] I could write and test [code] in the same page. [In $C_{Web+GPT}$, I] had to switch between code runner and the content” (P12)

Participants also expressed in surveys that they found code writing questions in C_{EDBook} to be more useful than in $C_{Web+GPT}$, with significantly higher agreement with the survey question “The code writing questions helped me improve my understanding of the material” ($p < 0.05$). This might also be due to EDBooks giving richer feedback to code writing responses. In $C_{Web+GPT}$, several participants sought richer feedback by copying and pasting quiz questions into ChatGPT rather than scrolling to see the answer.

4.5.3 Participants found it easier to phrase/ask questions in C_{EDBook} than $C_{Web+GPT}$. Although ChatGPT in $C_{Web+GPT}$ was provided the context of the content being discussed, this was a *global* context. However, participants felt that in $C_{Web+GPT}$, it could be burdensome to provide the *local* context of their queries. Further, combining LLM input with content made P13 more confident that the LLM was prompted with the appropriate context:

“I know you said like in this one ($C_{Web+GPT}$) you had prompted it with all the information that it would need, but it was harder. [...] like I wanted to ask a specific question about this function, and it kind of knew which function I meant. But it was like assuming the function, so I didn’t know if it knew what I was referencing. So in this one (EDBook) I was a lot more confident that I was like: in this example, what do you mean by this thing? And there’s more one-to-one correlation of the question versus the information I’m saying.” (P13)

This local context could be crucial for understanding how to accurately respond to some of the brief questions that participants asked during our study, such as “what is url” (P20), “Can you give an example of it” (P15), or “what does cube mean” (P12). For some of these queries, ChatGPT might give accurate answers but without being confident about the local context, it can also respond with superfluous information. For example, “what is url” refers to a variable url in a code sample but without local context, ChatGPT respond to “what is url” by first describing what URLs are. One participant also expressed that they would feel more comfortable using LLMs through EDBooks in a classroom setting:

“I actually stay away from using like ChatGPT, because I feel like so much debate about it, and so many like cheating things related to it. [...] I feel like in an unregulated ChatGPT if I’m the one putting all the input in and not getting as personalized output, it feels a little bit risky like am I crossing the line? But with this (EDBook), like everything was prepared for me, [...] it felt a lot safer if was an assignment or something. I wouldn’t feel guilty about using it because it was created for me. [...] I really like the academic safety of it.” (P10)

4.5.4 The types of questions that participants asked in C_{EDBook} were different from those in $C_{Web+GPT}$. We found qualitative differences in the types of questions that participants asked in the two conditions. In $C_{Web+GPT}$, we observed more participants copying and pasting a local context from the content (4.5.3). Some participants in $C_{Web+GPT}$ also copied

Statement	Condition	Strongly Disagree	Somewhat Disagree	Neither Agree nor Disagree	Somewhat Agree	Strongly Agree
I understand all the concepts and codes in the tutorial.	$C_{Web+GPT}$	15.0%	5.0%		55.0%	25.0%
	C_{EDBook}	10.0%	5.0%		50.0%	35.0%
I am confident in writing functions similar to what I learned in the tutorial.	$C_{Web+GPT}$	15.0%	10.0%	25.0%	35.0%	15.0%
	C_{EDBook}		35.0%	5.0%	35.0%	25.0%
The tutorial explained the concept clearly without any confusion.	$C_{Web+GPT}$	10.0%	15.0%		50.0%	25.0%
	C_{EDBook}	10.0%			50.0%	40.0%
The tutorial is easy to follow and understand.	$C_{Web+GPT}$	20.0%	5.0%		30.0%	45.0%
	C_{EDBook}	5.0%	5.0%		30.0%	60.0%
The tutorial is well structured with a suitable length.	$C_{Web+GPT}$	10.0%	10.0%		35.0%	45.0%
	C_{EDBook}	5.0%			35.0%	60.0%
I was highly engaged in the tutorial when learning.	$C_{Web+GPT}$	15.0%	20.0%	10.0%		45.0%
	C_{EDBook}	5.0%	5.0%		40.0%	50.0%
The tutorial met my learning preferences and needs.	$C_{Web+GPT}$	5.0%	20.0%	15.0%	25.0%	35.0%
	C_{EDBook}	10.0%	20.0%		30.0%	40.0%
I found it easy to navigate through the content of the tutorial.	$C_{Web+GPT}$	5.0%	15.0%		35.0%	45.0%
	C_{EDBook}	5.0%			55.0%	40.0%
The code examples were easy to follow and understand.	$C_{Web+GPT}$	20.0%	5.0%		35.0%	40.0%
	C_{EDBook}	5.0%	10.0%		45.0%	40.0%
The multiple-choice questions helped me improve my understanding of the material.	$C_{Web+GPT}$	10.0%	10.0%		30.0%	50.0%
	C_{EDBook}	5.0%			40.0%	55.0%
The code-writing questions helped me improve my understanding of the material.	$C_{Web+GPT}$	15.0%	15.0%		20.0%	50.0%
	C_{EDBook}	10.0%	5.0%	5.0%		75.0%
Code pointers (which highlighted bits of code) were useful.	C_{EDBook}	5.0%	15.0%			80.0%
The ability to select between multiple possible messages was useful.	C_{EDBook}	5.0%	10.0%		30.0%	55.0%
The ability to write custom messages is useful.	C_{EDBook}	5.0%	20.0%		25.0%	50.0%

Table 4. Post-task questionnaire results for participants in both conditions.

and pasted quiz questions to get more feedback (4.5.2). We found that LLM queries in C_{EDBook} tended to focus more on applying conceptual knowledge and exploring edge cases of concepts. We also observed two participants in C_{EDBook} who used the LLM to have conversations that were more tangential to the learning content—for example, to describe their background.

4.5.5 Participants found it easier to navigate and search the contents in $C_{Web+GPT}$. Participants expressed that they found it easier to navigate and search tutorial content in $C_{Web+GPT}$. There were two primary reasons for this. First, our EDBook content was written as a dialog and thus contained some messages that advanced the dialog but were tangential to the learning goals. Some participants felt that this was superfluous:

“Some of the suggestions felt like a little bit fluffy. [...] I think they mostly care about learning language and just getting to it. [...] I think it could have been reduced just to get rid of those like fluffy questions, I would just rather get to the point.” (P10)

Second, the “find on page” and skimming interactions in the EDBook platform are not as refined as they are in commercial web browsers. This could make navigation more difficult for some participants:

“Whenever I have questions [in $C_{Web+GPT}$] I can just do a Ctrl+F and go back to the previous content. So it’s more free and it’s easier to navigate for me.” (P9)

Despite this interview feedback from some participants, there were not significant differences in responses to survey questions about the ease of navigation in C_{EDBook} compared to $C_{\text{Web+GPT}}$.

4.5.6 *Participants found C_{EDBook} to be more engaging.* When asked to compare C_{EDBook} with $C_{\text{Web+GPT}}$, most (13) participants independently expressed that they found EDBooks to be more engaging than ChatGPT (our interview question did not directly ask about engagement). Some representative quotes from participants:

“So I’m a lot more engaged for [C_{EDBook}], but [in $C_{\text{Web+GPT}}$] I kind of just lost track of what I was reading.” (P1)

“I think I kind of prefer [C_{EDBook}] learning experience. I think, like more engaging and interactive.” (P4)

“The part that I like about [C_{EDBook}] is that the interactive element is helping me keep me stay engaged with the materials.” (P5)

“I like the dialogue style, because I feel like, I mean, obviously, I’m aware that it’s not a real person, but I think it’s I like the way that the instructor is talking, and it’s kind of more engaging than just like reading the text.” (P13)

This increased engagement is also supported by our survey results, where participants’ self-rated agreement with the statement “I was highly engaged in the tutorial.” was significantly higher in C_{EDBook} than $C_{\text{Web+GPT}}$ ($p < 0.05$). However, two participants disagreed with this sentiment, expressing that pre-written responses discouraged them from engaging in critical thinking:

“I become lazy—I know it’ll always give me a suggested message.” (P9)

4.5.7 *Participants would prefer EDBook content when learning something for the first time and a standard LLM UI when they were familiar with the material.* In post-task interviews, we asked participants to speculate on when they would prefer EDBook content and when they would prefer ‘standard’ content and LLMs. The most common answer (from nine participants) is that they would prefer EDBooks when learning material for the first time and ‘standard’ LLM interactions when they are already familiar with the material:

“[EDBooks] will be helpful when I’m learning a language for the first time but [ChatGPT] will be better when I already have prior knowledge in it” (P2)

“If I were a new student like my first year of programming, if I had something like [EDBooks], I would really like it. Because learning it the traditional way was quite difficult for someone who’s starting, but if I had this on day one, I would use it a lot.” (P15)

This feedback is also supported by the survey results, where participants responded that EDBook content was easy to follow and found the quiz questions more useful in C_{EDBook} . For $\mathcal{T}_{\text{cache}}$, C_{EDBook} had significantly higher ratings for both questions than $C_{\text{Web+GPT}}$. It is also supported by related interview responses, where participants expressed that EDBooks were more engaging (4.5.6).

4.6 User Evaluation — Limitations and Discussion

Our results provide important insight into how participants interact with educational content (both EDBooks and “standard” content with ChatGPT). However, as with any controlled study, there are important limitations to consider with respect to the results. First, our participant pool is not representative of the demographics of all learners. Second, participant feedback to EDBook content might have been more positive for participants who wanted to play the role

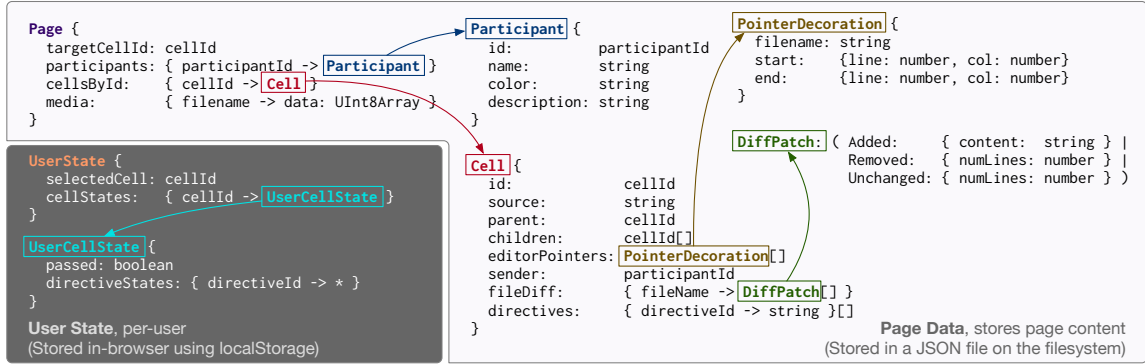


Fig. 8. An illustration of how EDBook content is represented and stored. We differentiate between *page data*—which stores the content of the page and is stored in a file—and *user data*—which represents user-specific information (such as which multiple choice options they selected or which message is currently visible) that is stored in ephemeral local storage.

of a “good participant”. Finally, our study was a short term study and is not necessarily representative of longer-term usage patterns.

Still, we believe our results are indicative of realistic interaction patterns. Overall, we found that participants appreciated many of the interactive parts of EDBooks and found them more engaging. In our study, this increased engagement did not lead to improved assessment scores (assessment score averages in C_{EDBook} were higher and time spent on assessment in C_{EDBook} was lower but neither difference with $C_{\text{Web+GPT}}$ was statistically significant). However, many prior studies have found that increased learner engagement improves learning outcomes [16].

Participants also pointed to ways that the EDBook platform could improve: (1) improving navigability and (2) promoting critical thinking. We believe that navigability (1) could be improved with relatively small implementation changes—writing more concise dialogic content and engineering better “search” features. However, promoting critical thinking (2) might require more substantial design changes, as we will discuss in section 6.3. Some of the challenges participants faced in $C_{\text{Web+GPT}}$ could also be addressed with design changes. For example, several browser extensions that integrate LLMs with content (for example, by clicking a portion of content to provide it as context to an inline chat window) could make it easier to provide local context.

5 IMPLEMENTATION

EDBook is implemented as a plugin for Visual Studio Code, built using TypeScript, React, and Redux⁴. Content is stored as a JavaScript Object Notation (JSON) file with the extension `.dpage`.

5.1 Page Format

Figure 8 illustrates the structure of EDBook content files (JSON data with a `.dpage` extension). The format of EDBook content was inspired by that of Jupyter’s `.ipynb` Notebooks [31]. However, some notable differences with Jupyter’s format are (1) a tree structure for cells (rather than lists); (2) a notion of users and provenance to track who sent which message; (3) “pointers” that better link narratives with code; (4) code diffs that allow code to change as the narrative advances; (5) education-specific “directives” such as multiple-choice questions and coding questions; and (6) separating

⁴The (anonymized) source code for EDBook is available at <https://github.com/anonymizer/EDBook>

users' state from the page data. Every node in the dialog tree contains message text, speaker metadata, code pointers, and directive metadata like multiple choice questions.

User progress through dialog trees is maintained in browser local storage. This includes the current node, directive answers, and whether nodes were visited. Separating mutable user state from immutable page content allows pages to be shared between learners without passing on user-specific data. It also allows EDBooks to store user data locally, even for content that is deployed publicly.

5.2 Syntax, Custom Directives, and Extensibility

EDBook documents use Markdown, a popular and simple markup language. EDBook uses an extended version of the Markdown syntax that enables custom generic directives, which we use for interactive assessments. For example, a user can author a multiple-choice question:

```
:::multiple-choice
Which of the following adds up to `2`?
::option[5+5]{feedback="Not quite. That adds to 10"}
::option[1+1]{correct}
::option[2+2]{feedback="Not quite. That adds to 4"}
:::
```

Whenever an author updates a message's content, EDBook scans the message for any custom directives (including multiple-choice and code writing). It then creates a unique ID for each directive by concatenating the message's ID with the directive type (e.g., `multiple-choice`) and the index of that directive. This way, users do not need to specify unique IDs manually [11] and these IDs are resilient to wording changes and other small modifications. This unique directive ID is used as a lookup key to store and recall the user's state for that directive (e.g., which multiple choice options are selected or the user's current code). Although the EDBook platform does not have a formal plugin API, adding new directives only involves defining a React or JavaScript component that can (1) render HTML output, (2) serialize the component state, and (3) subsequently load that serialized state. Everything else (including tracking multiple directives' states, determining when to render directives, and more) is handled by the EDBook platform. In the current implementation, directive states are entirely local and self-contained. This means, for example, that a multiple-choice question could not render differently depending on a user's response to a different question.

5.3 Tracking Larger Code Samples, Code Progression, and Incremental Edits

Every message can have any number of larger associated code samples that get displayed when that cell is selected. These code samples are stored separately from the cell source. Authors can write code samples by simply selecting a cell and updating the code (e.g., adding files, removing files, or editing file content). Because these code samples can be lengthy and are likely to remain unchanged between multiple cells, the EDBook platform stores code *changes* between cells, rather than the full code. This helps reduce the file size of the resulting .dpage files. As Figure 8 illustrates, every message (cell) contains a list of textual diffs. These represent additions, removals, and unmodified lines relative to prior versions. Whenever a modification that may change the code is made (e.g., a cell is moved or deleted; or the code associated with a cell is changed), the code state for every cell is re-constructed and the code diffs are re-derived.

5.4 Other Implementation Details

OpenAI’s ChatGPT Application Programming Interface (API) powers contextual LLM interactions. Readers can choose which model to query (GPT-3.5 by default). When users query the LLM, the current dialog context is combined with their question to generate a response. Constraining LLM prompts with dialog context mitigates unhelpful responses.

Uploaded images and other media are converted to binary data (UInt8Array), associated with a filename, and stored as part of the .dpage file (in the media field, as Figure 8 illustrates). Any deictic code references are stored as additional cell metadata, separate from the cell’s source.

6 LIMITATIONS AND FUTURE WORK

Our design of EDBook and its evaluation have several limitations that provide opportunities for future work.

6.1 Encouraging Meaningful Exploration and Reducing Manual Work

As Figure 2 illustrates, EDBooks exist in a continuum between less constrained tools (e.g., open-ended LLMs) and more constrained tools (e.g., pre-written linear dialogs). Although its design enables freeform user queries, the design nudges readers towards the pre-written narrative rather than self-directed exploration. EDBooks were designed this way partially because of concerns about the accuracy and usefulness of information generated by LLMs—we prioritize presenting messages that we can guarantee to be accurate and pedagogically productive. As LLM capabilities (and our trust in them) continue to improve, future systems could instead consider nudging readers towards open-ended interactions and emphasizing LLM-generated content. Future work could also involve fine-tuning LLMs to improve their ability to generate accurate and contextually relevant outputs that align with specific learning goals. Further, the EDBook design could also be adapted to use LLMs to generate additional assessments and interactive content.

Further, although content authors can use LLMs to pre-generate content, we found that authoring branched narratives and coding challenges requires significant effort. Even if authors use LLMs, it can take significant effort to engineer prompts that produce high-quality content. Semi-automating content generation through AI could make authoring more scalable. However, safety mechanisms will also be needed to address potential issues around bias, misinformation, and harmful content from LLMs.

6.2 Collaborative Features

In its current design, readers navigate EDBook content individually. However, readers would likely benefit and learn from other readers’ inquiries and interactions with EDBook content. For example, future versions could give readers the option to share their open-ended interactions with other learners. These shared user-generated narratives could be curated to ensure that their content is appropriate, accurate, and high-quality. Reputation systems could surface high-quality user contributions and community voting or experts could identify helpful perspectives. User-generated content might even be able to ultimately replace the original dialog-tree to create community-written narratives that continue to evolve and improve over time.

6.3 Generalizing Beyond Programming

Many aspects of the EDBook platform’s design could apply to domains beyond programming. For example, dialogs could guide readers through advanced UIs or creative and open-ended domains. To start, future work could replace

EDBook’s code panel with applications that are appropriate for the given domain. With different prompting strategies, users could author customized tutorials for their own domain of work.

7 CONCLUSION

Programming education at scale remains a vital challenge. Traditional materials often lack interactivity, guidance, and engagement. Although learning with open-ended LLMs have many benefits, it also has certain negative impacts such as unreliable responses and a lack of a clear learning goal. This work presents EDBook, a platform that combines goal-aligned dialog trees with open-ended LLM interactions. We describe the design and implementation of tree-based dialog management, goal alignment techniques, and personalized programming exercises. Our user study demonstrates several key benefits when using EDBooks versus traditional materials. Navigating the balance between open-ended LLM interactions and pedagogical goals is still an ongoing research challenge. Learners benefited from the conversational agents to pose questions, but structured guidance was needed to maintain educational focus. Our work contributes to the field of human-centric AI in programming education by demonstrating promising methodologies. We see great potential for dialogic learning at scale through AI. The design insights from EDBook point towards more engaging, responsive, and human-aligned programming education with AI.

REFERENCES

- [1] AL-GAHMI, A., ZHANG, Y., AND VALLE, H. Jupyter in the classroom: An experience report. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education - Volume 1* (2022), p. 425–431.
- [2] BECKER, B. A., DENNY, P., FINNIE-ANSLEY, J., LUXTON-REILLY, A., PRATHER, J., AND SANTOS, E. A. Programming is hard-or at least it used to be: Educational opportunities and challenges of ai code generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (2023), pp. 500–506.
- [3] BIBAUW, S., VAN DEN NOORTGATE, W., FRANÇOIS, T., AND DESMET, P. Dialogue systems for language learning: A meta-analysis. *Language Learning & Technology* 26, 1 (2022).
- [4] BLOOM, B. Bloom’s taxonomy, 1956.
- [5] BUBECK, S., CHANDRASEKARAN, V., ELKAN, R., GEHRKE, J., HORVITZ, E., KAMAR, E., LEE, P., LEE, Y. T., LI, Y., LUNDBERG, S., ET AL. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712* (2023).
- [6] CHEN, M., TWOREK, J., JUN, H., YUAN, Q., PINTO, H. P. D. O., KAPLAN, J., EDWARDS, H., BURDA, Y., JOSEPH, N., BROCKMAN, G., ET AL. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [7] COMMUNITY, E. B. Jupyter book, Feb. 2020.
- [8] CRANSHAW, J., ELWANY, E., NEWMAN, T., KOCIELNIK, R., YU, B., SONI, S., TEEVAN, J., AND MONROY-HERNÁNDEZ, A. Calendar. help: Designing a workflow-based scheduling agent with humans in the loop. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (2017), pp. 2382–2393.
- [9] DAHL, D. A. Visualization tools for designing spoken dialogs. *Practical spoken dialog systems* (2005).
- [10] DENNY, P., LUXTON-REILLY, A., AND SIMON, B. Evaluating a new exam question: Parsons problems. In *Proceedings of the fourth international workshop on computing education research* (2008), pp. 113–124.
- [11] ERICSON, B. J., AND MILLER, B. N. Runestone: A platform for free, on-line, and interactive ebooks. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (2020), pp. 1012–1018.
- [12] FÄRNQVIST, T., HEINTZ, F., LAMBRIX, P., MANNILA, L., AND WANG, C. Supporting active learning by introducing an interactive teaching tool in a data structures and algorithms course. In *Proceedings of the 47th ACM Technical Symposium on Computing Science Education* (2016), p. 663–668.
- [13] FRIEDMAN, D. P., AND FELLEISEN, M. *The Little Schemer*. MIT Press, 1995.
- [14] FRIEDMAN, D. P., AND FELLEISEN, M. *The seasoned schemer*. MIT Press, 1995.
- [15] FRIJTERS, S., TEN DAM, G., AND RIJLAARSDAM, G. Effects of dialogic learning on value-loaded critical thinking. *Learning and Instruction* 18, 1 (2008), 66–82.
- [16] GRAY, J. A., AND DILORETO, M. The effects of student engagement, student satisfaction, and perceived learning in online learning environments. *International Journal of Educational Leadership Preparation* 11, 1 (2016), n1.
- [17] GRUDIN, J., AND JACQUES, R. Chatbots, humbots, and the quest for artificial general intelligence. In *Proceedings of the 2019 CHI conference on human factors in computing systems* (2019), pp. 1–11.
- [18] GUO, P. J. Six opportunities for scientists and engineers to learn programming using ai tools such as chatgpt.

- [19] HAN, X., ZHOU, M., TURNER, M. J., AND YEH, T. Designing effective interview chatbots: Automatic chatbot profiling and design suggestion generation for chatbot debugging. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (2021), pp. 1–15.
- [20] HEAD, A., JIANG, J., SMITH, J., HEARST, M. A., AND HARTMANN, B. Composing flexibly-organized step-by-step tutorials from linked source code, snippets, and outputs. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (2020), pp. 1–12.
- [21] HEAVEN, W. Chatgpt is going to change education, not destroy it, 2023.
- [22] HICKE, Y., AGARWAL, A., MA, Q., AND DENNY, P. Chata: Towards an intelligent question-answer teaching assistant using open-source llms. *arXiv preprint arXiv:2311.02775* (2023).
- [23] HIRSH-PASEK, K., AND BLINKOFF, E. Chatgpt: educational friend or foe?
- [24] JAIN, M., KUMAR, P., KOTA, R., AND PATEL, S. N. Evaluating and informing the design of chatbots. In *Proceedings of the 2018 designing interactive systems conference* (2018), pp. 895–906.
- [25] JELL, L., LIST, C., AND KIPP, M. Towards automated interactive tutoring-focussing on misconceptions and adaptive level-specific feedback. In *Proceedings of the 5th European Conference on Software Engineering Education* (2023), pp. 226–235.
- [26] JIANG, P., RAYAN, J., DOW, S. P., AND XIA, H. Graphologue: Exploring large language model responses with interactive diagrams. *arXiv preprint arXiv:2305.11473* (2023).
- [27] JOHNSON, J. W. Benefits and pitfalls of jupyter notebooks in the classroom. In *Proceedings of the 21st Annual Conference on Information Technology Education* (2020), p. 32–37.
- [28] JOYNER, D. A. Chatgpt in education: Partner or pariah? *XRDS: Crossroads, The ACM Magazine for Students* 29, 3 (2023), 48–51.
- [29] KAZEMITABAAR, M., CHOW, J., MA, C. K. T., ERICSON, B. J., WEINTROP, D., AND GROSSMAN, T. Studying the effect of ai code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (2023), pp. 1–23.
- [30] KIM, T. S., LEE, Y., CHANG, M., AND KIM, J. Cells, generators, and lenses: Design framework for object-oriented interaction with large language models.
- [31] KLUYVER, T., RAGAN-KELLEY, B., PÉREZ, F., GRANGER, B. E., BUSSONNIER, M., FREDERIC, J., KELLEY, K., HAMRICK, J. B., GROUT, J., CORLAY, S., ET AL. Jupyter notebooks-a publishing format for reproducible computational workflows. *Elpub* 2016 (2016), 87–90.
- [32] LEE, C., JUNG, S., EUN, J., JEONG, M., AND LEE, G. G. A situation-based dialogue management using dialogue examples. In *2006 IEEE International Conference on Acoustics Speech and Signal Processing Proceedings* (2006), vol. 1, IEEE, pp. I–I.
- [33] LEINONEN, J., DENNY, P., MACNEIL, S., SARSA, S., BERNSTEIN, S., KIM, J., TRAN, A., AND HELLAS, A. Comparing code explanations created by students and large language models. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (2023), p. 124–130.
- [34] LEINONEN, J., HELLAS, A., SARSA, S., REEVES, B., DENNY, P., PRATHER, J., AND BECKER, B. A. Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (2023), p. 563–569.
- [35] LI, C.-H., YEH, S.-F., CHANG, T.-J., TSAI, M.-H., CHEN, K., AND CHANG, Y.-J. A conversation analysis of non-progress and coping strategies with a banking task-oriented chatbot. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (2020), pp. 1–12.
- [36] LI, J., TAN, C. W., HANG, C. N., AND QI, X. A chatbot-server framework for scalable machine learning education through crowdsourced data. In *Proceedings of the Ninth ACM Conference on Learning@ Scale* (2022), pp. 271–274.
- [37] LI, Y., CHOI, D., CHUNG, J., KUSHMAN, N., SCHRITTWIESER, J., LEBLOND, R., ECCLES, T., KEELING, J., GIMENO, F., LAGO, A. D., HUBERT, T., CHOY, P., DE MASSON D’AUTUME, C., BABUSCHKIN, I., CHEN, X., HUANG, P.-S., WELBL, J., GOWAL, S., CHEREPANOV, A., MOLLOY, J., MANKOWITZ, D. J., ROBSON, E. S., KOHLI, P., DE FREITAS, N., KAVUKCUOGLU, K., AND VINYALS, O. Competition-level code generation with alphacode. *Science* 378, 6624 (2022), 1092–1097.
- [38] LIFFITON, M., SHEESE, B., SAVELKA, J., AND DENNY, P. Codehelp: Using large language models with guardrails for scalable support in programming classes. *arXiv preprint arXiv:2308.06921* (2023).
- [39] LIN, X. Exploring the role of chatgpt as a facilitator for motivating self-directed learning among adult learners. *Adult Learning* (2023), 10451595231184928.
- [40] LLC, G. Dialogflow, 2023. Accessed: 2023-09-01.
- [41] LU, X., FAN, S., HOUGHTON, J., WANG, L., AND WANG, X. Readingquizmaker: A human-nlp collaborative system that supports instructors to design high-quality reading quiz questions. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (2023), pp. 1–18.
- [42] LUO, C. J., WONG, V. Y. L., AND GONDA, D. E. Code free chatbot development: An easy way to jumpstart your chatbot! In *Proceedings of the Seventh ACM Conference on Learning@ Scale* (2020), pp. 233–235.
- [43] MARKEL, J. M., OPFERMAN, S. G., LANDAY, J. A., AND PIECH, C. Gpteach: Interactive ta training with gpt based students.
- [44] MILLER, B. N., AND RANUM, D. L. Beyond pdf and epub: Toward an interactive textbook. In *Proceedings of the 17th ACM Annual Conference on Innovation and Technology in Computer Science Education* (2012), p. 150–155.
- [45] MUISE, C., CHAKRABORTI, T., AGARWAL, S., BAJGAR, O., CHAUDHARY, A., LASTRAS-MONTANO, L. A., ONDREJ, J., VODOLAN, M., AND WIECHA, C. Planning for goal-oriented dialogue systems. *arXiv preprint arXiv:1910.08137* (2019).
- [46] ONEY, S., BROOKS, C., AND RESNICK, P. Creating guided code explanations with chat. codes. *Proceedings of the ACM on Human-Computer Interaction* 2, CSCW (2018), 1–20.
- [47] PEARCE, H., AHMAD, B., TAN, B., DOLAN-GAVITT, B., AND KARRI, R. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)* (2022), pp. 754–768.
- [48] REEVES, B., SARSA, S., PRATHER, J., DENNY, P., BECKER, B. A., HELLAS, A., KIMMEL, B., POWELL, G., AND LEINONEN, J. Evaluating the performance

- of code generation models for solving parsons problems with small prompt variations. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1* (2023), p. 299–305.
- [49] RESNICK, L. B., ASTERHAN, C. S., CLARKE, S. N., AND SCHANTZ, F. Next generation research in dialogic learning. *Wiley handbook of teaching and learning* (2018), 323–338.
- [50] SANTOS TEIXEIRA, M., AND DRAGONI, M. A review of plan-based approaches for dialogue management. *Cognitive Computation* 14, 3 (2022), 1019–1038.
- [51] SARSA, S., DENNY, P., HELLAS, A., AND LEINONEN, J. Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research - Volume 1* (2022), p. 27–43.
- [52] SHAFFER, C. A., KARAVIRTA, V., KORHONEN, A., AND NAPS, T. L. Opensda: Beginning a community active-ebook project. In *Proceedings of the 11th Koli Calling International Conference on Computing Education Research* (2011), p. 112–117.
- [53] SMITH, D. H., HAO, Q., HUNDHAUSEN, C. D., JAGODZINSKI, F., MYERS-DEAN, J., AND JAEGER, K. Towards modeling student engagement with interactive computing textbooks: An empirical study. In *Proceedings of the 52nd ACM Technical Symposium on Computer Science Education* (2021), p. 914–920.
- [54] SUH, S., MIN, B., PALANI, S., AND XIA, H. Sensecape: Enabling multilevel exploration and sensemaking with large language models. *arXiv preprint arXiv:2305.11483* (2023).
- [55] TANG, J., ZHAO, T., XIONG, C., LIANG, X., XING, E. P., AND HU, Z. Target-guided open-domain conversation. *arXiv preprint arXiv:1905.11553* (2019).
- [56] TANPRASERT, T., FELS, S. S., SINNAMON, L., AND YOON, D. Scripted vicarious dialogues: Educational video augmentation method for increasing isolated students’ engagement. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (2023), pp. 1–25.
- [57] TEAM, C. Character.ai, 2023. Accessed: 2023-09-01.
- [58] TEO, P. Teaching for the 21st century: A case for dialogic pedagogy. *Learning, Culture and Social Interaction* 21 (2019), 170–178.
- [59] WANG, A. Y., AND CHILANA, P. K. Designing curated conversation-driven explanations for communicating complex technical concepts. In *2019 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)* (2019), IEEE, pp. 211–215.
- [60] WANG, A. Y., HEAD, A., ZHANG, A. G., ONEY, S., AND BROOKS, C. Colaroid: A literate programming approach for authoring explorable multi-stage tutorials. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (2023), pp. 1–22.
- [61] WANG, A. Y., WU, Z., BROOKS, C., AND ONEY, S. Callisto: Capturing the “why” by connecting conversations with computational narratives. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (2020), pp. 1–13.
- [62] WEGERIF, R. *Dialogic education and technology: Expanding the space of learning*, vol. 7. Springer Science & Business Media, 2007.
- [63] WOOD, A. K., GALLOWAY, R. K., SINCLAIR, C., AND HARDY, J. Teacher-student discourse in active learning lectures: case studies from undergraduate physics. *Teaching in Higher Education* 23, 7 (2018), 818–834.
- [64] XIE, T., YANG, X., LIN, A. S., WU, F., HASHIMOTO, K., QU, J., KANG, Y. M., YIN, W., WANG, H., YAVUZ, S., ET AL. Converse: A tree-based modular task-oriented dialogue system. *arXiv preprint arXiv:2203.12187* (2022).
- [65] ZAMORA, J. I’m sorry, dave, i’m afraid i can’t do that: Chatbot perception and expectations. In *Proceedings of the 5th international conference on human agent interaction* (2017), pp. 253–260.